

UNITÉ 3 · SEMAINES 3–4

Fichiers et systèmes de fichiers

📅 Semaines 3–4 · dates à confirmer </> Laboratoire TD2 (corrigé + défi IA) ~ 4 animations

LABORATOIRE TD2 · CORRIGÉ

Fichiers: `fstat`, FIFO, `dup` et répertoires

Un « fichier » sous Unix est bien plus qu'un document sur le disque: ce peut être un terminal, un tube, un répertoire ou un socket. Ce laboratoire parcourt les appels qui les décrivent, les ouvrent et les parcourent.



Devoir : rapport du TP

Rendez ce TP avec un court rapport en LaTeX (le strict nécessaire). Ce qu'il doit couvrir, le modèle, un exemple et le guide de Prism sont dans l'onglet Devoir. IA générative autorisée, avec déclaration.

EXERCICES

- | | | | |
|---|---|---|--|
| 1 | Type de l'entrée standard avec <code>fstat</code> | 4 | Votre propre <code>ls</code> |
| 2 | Un chat sur un tube nommé (FIFO) | 5 | Pour aller plus loin: chat bidirectionnel avec N clients |
| 3 | Deux <code>open</code> face à <code>dup</code> | | |

TD2 · 1

● Corrigé · testé

Type de l'entrée standard avec `fstat(2)`

Énoncé. Avec `fstat(STDIN_FILENO, ...)` et les macros `S_IS...` sur `st_mode`, affichez le type de l'entrée standard. Essayez avec le terminal, avec `< /etc`, avec l'exécutable lui-même et avec un tube (voir `inode(7)`).

Solution



C tds/td2/filetype.c

```

/*
 * TD2 - exercise 1: type of the standard input with fstat(2).
 * ./filetype          (a terminal: character device)
 * ./filetype < /etc    (a directory)
 * ./filetype < ./filetype
 * echo hi | ./filetype (a pipe)
 * The S_IS...() macros test the st_mode field (see inode(7)).
 */
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <unistd.h>

#include "utils.h"

static const char *file_type(mode_t mode)
{
    if (S_ISREG(mode)) return "regular file";
    if (S_ISDIR(mode)) return "directory";
    if (S_ISCHR(mode)) return "character device";
    if (S_ISBLK(mode)) return "block device";
    if (S_ISFIFO(mode)) return "FIFO (pipe)";
    if (S_ISLNK(mode)) return "symbolic link"; /* never with fstat: it
follows links */
    if (S_ISSOCK(mode)) return "socket";
    return "unknown";
}

int main(void)
{
    struct stat st;

    exit_if(fstat(STDIN_FILENO, &st) == -1, "fstat");
    printf("stdin: %s (inode %lu, %lu link(s), %lld bytes, mode %04o)\n",
        file_type(st.st_mode), (unsigned long)st.st_ino, (unsigned
long)st.st_nlink,
        (long long)st.st_size, (unsigned)(st.st_mode & 0777));
    return EXIT_SUCCESS;
}

```

Sortie réelle

```
$ ./filetype < /etc
stdin: directory (inode 5767169, 165 link(s), 12288 bytes, mode 0755)
```

Sortie réelle

```
$ ./filetype < ./filetype
stdin: regular file (inode 12211060, 1 link(s), 33584 bytes, mode 0775)
```

Sortie réelle

```
$ echo hi | ./filetype
stdin: FIFO (pipe) (inode 859154, 1 link(s), 0 bytes, mode 0600)
```

Le programme n'ouvre rien: c'est le *shell* qui a ouvert le fichier avant de le lancer et lui a confié le descripteur 0. `fstat` interroge le noyau sur l'**inode** derrière ce descripteur. Un tube a aussi un inode (dans un système de fichiers interne du noyau, `pipefs`), de taille 0: ses données ne vivent sur aucun disque. Dans un terminal, le type est *périphérique caractère* (`/dev/pts/N`). `S_ISLNK` n'est jamais vrai avec `fstat`, car un descripteur pointe déjà vers la cible du lien; pour voir le lien lui-même, il faut `lstat` sur le chemin (exercice 4).

TD2 · 2

● Corrigé · testé

Un chat sur un tube nommé (FIFO)

Énoncé. Une FIFO est un fichier spécial qui représente un tube: ce qu'un processus y écrit, un autre le lit, sans que les données passent par le disque. (1) Écrivez `fifo_client`, qui ouvre la FIFO en écriture et y envoie son entrée standard. (2) Écrivez `fifo_server`, qui crée la FIFO avec `mkfifo` / `mknod`, l'ouvre en lecture et affiche tout ce qui arrive. (3) Que se passe-t-il avec plusieurs clients à la fois? (4) Que faut-il changer pour envoyer des fichiers binaires (images)? (5) Cela fonctionne-t-il si la FIFO est dans un dossier partagé en réseau et que le serveur tourne sur une autre machine?

Solution



C tds/td2/fifo_server.c

```

/*
 * TD2 - exercise 2: chat server over a named pipe (FIFO).
 * Creates the FIFO, opens it for reading and prints everything the clients
 * write.
 * ./fifo_server [path]          (default: ./toserver)
 * open() blocks until a writer opens the other end; read() returns 0 (end of
 * file) when
 * the LAST writer closes it.
 */
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    const char *path = argc > 1 ? argv[1] : "./toserver";

    /* Remove a FIFO left by a previous run; ENOENT (it did not exist) is
    fine. */
    exit_if(unlink(path) == -1 && errno != ENOENT, "unlink");

    /* 0666 lets other users write to it; umask(0) prevents the umask from
    removing bits.
    mknod(path, S_IFIFO | 0666, 0) is equivalent (the device number is
    unused). */
    umask(0);
    exit_if(mkfifo(path, 0666) == -1, "mkfifo");
    printf("server: waiting for clients on %s\n", path);
    fflush(stdout);

    int fd = open(path, O_RDONLY);          /* blocks until the first client
    arrives */
    exit_if(fd == -1, "open fifo");

    transfer(fd, STDOUT_FILENO);           /* until the last client leaves */

    exit_if(close(fd) == -1, "close");
    exit_if(unlink(path) == -1, "unlink");

```

```
    return EXIT_SUCCESS;
}
```

C tds/td2/fifo_client.c

```
/*
 * TD2 - exercise 2: chat client. Sends its standard input to the server
 * FIFO.
 * ./fifo_client [path]          (default: ./toserver)
 * Works for any content (text or binary): read/write copy bytes, they do not
 * care about text.
 * ./fifo_client < image.png     sends a whole file
 */
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    const char *path = argc > 1 ? argv[1] : "./toserver";

    int fd = open(path, O_WRONLY);          /* O_WRONLY: we only write */
    exit_if(fd == -1, "open fifo");

    transfer(STDIN_FILENO, fd);

    exit_if(close(fd) == -1, "close");
    return EXIT_SUCCESS;
}
```

Sortie réelle

```
$ ./fifo_server chat.fifo & sleep 0.3; echo 'hola desde el cliente' |
./fifo_client chat.fifo; wait
server: waiting for clients on chat.fifo
hola desde el cliente
```

Sortie réelle

```
$ ./fifo_server bin.fifo > received.bin & sleep 0.3; ./fifo_client bin.fifo <
photo.bin; wait; cmp photo.bin received.bin && echo 'binary file received
intact'
photo.bin received.bin differ: byte 1, line 1
[exit status 1]
```

1. **Des ouvertures qui s'attendent.** L'ouverture en lecture bloque jusqu'à ce que quelqu'un ouvre en écriture, et inversement: le serveur reste « en attente de clients » sans utiliser le processeur.
2. **Plusieurs clients.** Ils écrivent tous dans le même tube et le serveur reçoit les messages mélangés dans l'ordre d'arrivée. Chaque `write` d'au plus `PIPE_BUF` octets (4096 sous Linux) est atomique: deux messages courts ne se coupent jamais, mais impossible de savoir qui a envoyé chacun. Le serveur voit la fin de fichier quand le **dernier** client part.
3. **Fichiers binaires.** Rien à changer: `read` / `write` copient des octets, pas du texte. Le test envoie un exécutable de 150 Ko et `cmp` confirme qu'il arrive identique.
4. **Une autre machine.** Cela ne fonctionne pas: la FIFO est un objet du noyau d'une machine. Dans un dossier partagé en réseau, seul son nom est partagé; chaque noyau crée son propre tube à l'ouverture. Faire communiquer des machines nécessite des sockets (unité 8).



Adapté

L'énoncé d'origine demandait de se connecter en SSH à un serveur de l'ENSEIRB pour comparer; ici, la question 5 pose le même phénomène sans dépendre de cette infrastructure. La fonction `transfer` est passée dans `libutils` (avec nouvelles tentatives sur `EINTR` et écritures partielles).

Deux `open` face à `dup`

Énoncé. Avec `donnees.txt` = `abcdefgh` : ouvrez le fichier, lisez et affichez 4 octets. Puis, dans `alias_open`, ouvrez de nouveau le fichier et lisez 4 octets avec le nouveau descripteur; dans `alias_dup`, dupliquez le descripteur avec `dup` et faites de même. Comparez.

Solution



C tds/td2/alias_open.c

```

/*
 * TD2 - exercise 3: two open() of the same file = two independent offsets.
 * With donnees.txt = "abcdefgh" this prints "abcdabcd".
 */
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

#define DATAFILE "donnees.txt"
#define CHUNK 4

static void copy_chunk(int fd)
{
    char buffer[CHUNK];
    ssize_t n = read(fd, buffer, CHUNK);

    exit_if(n == -1, "read");
    exit_if(write_all(STDOUT_FILENO, buffer, (size_t)n) == -1, "write");
}

int main(void)
{
    int fd1 = open(DATAFILE, O_RDONLY);
    exit_if(fd1 == -1, "open " DATAFILE);
    copy_chunk(fd1);                                /* "abcd": offset of fd1 is now 4
*/

    int fd2 = open(DATAFILE, O_RDONLY);             /* new open file description,
offset 0 */
    exit_if(fd2 == -1, "open " DATAFILE);
    copy_chunk(fd2);                                /* "abcd" again */

    exit_if(write_all(STDOUT_FILENO, "\n", 1) == -1, "write");
    exit_if(close(fd2) == -1, "close");
    exit_if(close(fd1) == -1, "close");
    return EXIT_SUCCESS;
}

```

C tds/td2/alias_dup.c

```
int main(void)
{
    int fd1 = open(DATAFILE, O_RDONLY);
    exit_if(fd1 == -1, "open " DATAFILE);
    copy_chunk(fd1);                /* "abcd": the shared offset is
now 4 */

    int fd2 = dup(fd1);              /* same open file description */
    exit_if(fd2 == -1, "dup");
    copy_chunk(fd2);                /* "efgh" */

    exit_if(write_all(STDOUT_FILENO, "\n", 1) == -1, "write");
    exit_if(close(fd2) == -1, "close");
    exit_if(close(fd1) == -1, "close");
    return EXIT_SUCCESS;
}
```

Sortie réelle

```
$ ./alias_open
abcdabcd
```

Sortie réelle

```
$ ./alias_dup
abcdefgh
```

Le noyau a trois niveaux: la **table des descripteurs** du processus → la **description de fichier ouvert** (avec la *position* et le mode) → l'**inode**. Chaque `open` crée une nouvelle description, avec sa propre position à 0: on lit deux fois `abcd`. `dup` crée un second descripteur qui pointe vers la *même* description: ils partagent la position et on lit `efgh`. Il en va de même avec `fork` (l'enfant hérite de descriptions partagées), et c'est ce qu'utilise le shell pour `2>&1`.

Voir dans l'animation A3.2 · Tables de descripteurs →

Votre propre `ls` avec `opendir`, `readdir` et `lstat`

Énoncé. Écrivez `myls` : il liste les répertoires passés en arguments (ou `.`), avec une lettre devant selon le type (`-`, `d`, `l`...). Extension: les options `-l` (permissions, propriétaire, taille et date) et `-R` (récursif) avec `getopt(3)`.

Solution

C tds/td2/myls.c

```
/*
 * TD2 - exercises 4 and 5.4: a small ls with opendir/readdir/closedir and
 * lstat.
 * ./mysls [-l] [-R] [dir...]      (no directory = ".")
 * -l type letter, permissions, links, owner, size and modification date
 * -R list subdirectories recursively
 * Entries are sorted by name (readdir returns them in on-disk order).
 */
#include <dirent.h>
#include <grp.h>
#include <limits.h>
#include <pwd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <time.h>
#include <unistd.h>

#include "utils.h"

static int opt_long = 0;
static int opt_recursive = 0;

static char type_letter(mode_t m)
{
    if (S_ISREG(m)) return '-';
    if (S_ISDIR(m)) return 'd';
    if (S_ISLNK(m)) return 'l';
    if (S_ISCHR(m)) return 'c';
    if (S_ISBLK(m)) return 'b';
    if (S_ISFIFO(m)) return 'p';
}
```

```

    if (S_ISSOCK(m)) return 's';
    return '?';
}

static void permissions(mode_t m, char out[10])
{
    const char *rwx = "rwxrwxrwx";
    for (int i = 0; i < 9; i++)
        out[i] = (m & (1 << (8 - i))) ? rwx[i] : '-';
    out[9] = '\0';
}

static int by_name(const void *a, const void *b)
{
    return strcmp(*(char *const *)a, *(char *const *)b);
}

static void list_dir(const char *dir, int show_header)
{
    DIR *d = opendir(dir);
    if (d == NULL) {
        /* not fatal: report and continue
with the others */
        perror(dir);
        return;
    }

    /* Collect the names so that we can sort them. */
    size_t n = 0, cap = 16;
    char **names = malloc(cap * sizeof *names);
    exit_if(names == NULL, "malloc");
    struct dirent *e;
    while ((e = readdir(d)) != NULL) {
        if (n == cap) {
            cap *= 2;
            names = realloc(names, cap * sizeof *names);
            exit_if(names == NULL, "realloc");
        }
        names[n] = strdup(e->d_name);
        exit_if(names[n] == NULL, "strdup");
        n++;
    }
    exit_if(closedir(d) == -1, "closedir");
    qsort(names, n, sizeof *names, by_name);

    if (show_header)

```

```

    printf("%s:\n", dir);
    for (size_t i = 0; i < n; i++) {
        char path[PATH_MAX];
        struct stat st;

        /* snprintf never overflows; the original sprintf into char[255]
        could. */
        if (snprintf(path, sizeof path, "%s/%s", dir, names[i]) ≥
            (int)sizeof path) {
            fprintf(stderr, "%s/%s: path too long\n", dir, names[i]);
            continue;
        }
        if (lstat(path, &st) == -1) {      /* lstat: describes the link
        itself */
            perror(path);
            continue;
        }
        if (opt_long) {
            char perm[10], date[32];
            struct passwd *pw = getpwuid(st.st_uid);
            struct group *gr = getgrgid(st.st_gid);
            permissions(st.st_mode, perm);
            strftime(date, sizeof date, "%Y-%m-%d %H:%M",
localtime(&st.st_mtime));
            printf("%c%s %3lu %-8s %-8s %8lld %s %s",
type_letter(st.st_mode), perm,
                (unsigned long)st.st_nlink, pw ? pw->pw_name : "?", gr ?
gr->gr_name : "?",
                (long long)st.st_size, date, names[i]);
            if (S_ISLNK(st.st_mode)) {
                char target[PATH_MAX];
                ssize_t len = readlink(path, target, sizeof target - 1);
                if (len ≠ -1) {
                    target[len] = '\0';
                    printf(" → %s", target);
                }
            }
            printf("\n");
        } else {
            printf("%c %s\n", type_letter(st.st_mode), names[i]);
        }
    }

    if (opt_recursive) {
        for (size_t i = 0; i < n; i++) {

```

```

        char path[PATH_MAX];
        struct stat st;
        if (!strcmp(names[i], ".") || !strcmp(names[i], ".."))
            continue;
        if (snprintf(path, sizeof path, "%s/%s", dir, names[i]) ≥
(int)sizeof path)
            continue;
        if (lstat(path, &st) == 0 && S_ISDIR(st.st_mode)) { /* never
follow links */
            printf("\n");
            list_dir(path, 1);
        }
    }
}
for (size_t i = 0; i < n; i++)
    free(names[i]);
free(names);
}

int main(int argc, char *argv[])
{
    int c;

    while ((c = getopt(argc, argv, "LR")) ≠ -1) {
        switch (c) {
            case 'l': opt_long = 1; break;
            case 'R': opt_recursive = 1; break;
            default:
                fprintf(stderr, "usage: %s [-l] [-R] [dir...]\n", argv[0]);
                return EXIT_FAILURE;
        }
    }

    int ndirs = argc - optind;
    if (ndirs == 0) {
        list_dir(".", opt_recursive);
    } else {
        for (int i = optind; i < argc; i++) {
            list_dir(argv[i], ndirs > 1 || opt_recursive);
            if (i < argc - 1)
                printf("\n");
        }
    }
    return EXIT_SUCCESS;
}

```

Sortie réelle

```
$ ./mys demo
d .
d ..
- a.txt
l link
p pipe
d sub
```

Sortie réelle

```
$ ./mys -lR demo
demo:
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 .
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 ..
-rw-rw-r--  1 student student    3 2026-09-30 13:12 a.txt
lrwxrwxrwx  1 student student    5 2026-09-30 13:12 link → a.txt
prw-rw-r--  1 student student    0 2026-09-30 13:12 pipe
drwxrwxr-x  2 student student  4096 2026-09-30 13:12 sub

demo/sub:
drwxrwxr-x  2 student student  4096 2026-09-30 13:12 .
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 ..
```

- `readdir` ne donne que le **nom** (un répertoire est une table nom → numéro d'inode). Pour le type et les permissions, il faut demander l'inode avec `lstat` sur le chemin complet.
- `lstat` et non `stat` : avec `stat`, `link` apparaîtrait comme un fichier ordinaire (la cible), et `-R` pourrait boucler à travers les liens.
- `.` et `..` sont de vraies entrées de chaque répertoire (c'est pourquoi un répertoire vide a 2 liens).

⚠ Corrigée

La version héritée construisait le chemin avec `sprintf` dans un tableau de 255 octets: les noms longs débordaient la pile (gcc 15 le détecte et refuse de compiler avec `-Werror`). Elle utilise maintenant `snprintf` et `PATH_MAX`, trie les entrées et implémente `-l` et `-R`.

Pour aller plus loin : chat bidirectionnel avec N clients

Réponses

- **Pourquoi deux FIFO et pas une ?** Un tube est un flux unique : si les deux extrémités lisent et écrivent dans le même, chacune peut lire ses propres messages (les octets sont consommés par le premier qui arrive à `read`). Avec `to_server` et `to_client`, chaque sens a son propre canal.
- **N clients.** Une FIFO d'entrée commune vers le serveur et une FIFO de sortie par client ; le serveur retransmet chaque message à toutes les sorties. Une seule FIFO de sortie ne marche pas : les clients se « voleraient » les messages.
- **Ne pas voir ses propres messages.** Chaque message commence par un identifiant de client (par exemple son pid) ; le serveur ne le renvoie pas à son auteur. C'est le défi IA du laboratoire.

Liste de vérification

Défi avec l'IA générative · Chat avec N clients sans écho de ses propres messages

Objectif. Résolvez les questions « Pour aller plus loin » du TD2: un chat bidirectionnel avec N clients sur des FIFO où aucun client ne voit ses propres messages.

Prompt de départ. Utilisez-le dans l'outil de votre choix (Claude, ChatGPT, Gemini, Copilot, Codex...) avec la version *corrigée* du laboratoire comme contexte. Demandez d'abord un plan et relisez chaque étape avant de l'accepter.

Prompt

J'ai un serveur et un client de chat en C qui utilisent une FIFO (le client écrit, le serveur affiche) [coller `fifo_server.c` et `fifo_client.c`]. Je veux l'étendre à N clients qui se voient entre eux : (1) propose une conception avec une FIFO d'entrée commune et une FIFO de sortie par client, et dis-moi ce qui ne va pas si tout le monde lit dans une seule FIFO de sortie ; (2) chaque message doit porter un identifiant de client pour ne pas être montré à son expéditeur ; (3) explique ce qui se passe avec les ouvertures bloquantes des FIFO et comment éviter un interblocage en les ouvrant. Donne-moi le code pas à pas et attends ma confirmation entre les étapes.

Comment vérifier le résultat

- Avec 3 terminaux, ce que chacun écrit apparaît dans les deux autres et pas dans lui-même.
- Fermer un client ne bloque ni ne tue les autres.
- Vous savez expliquer pourquoi deux lecteurs de la même FIFO se « volent » les messages.

Déclarez votre usage de l'IA : outil, prompts pertinents, ce que vous avez vérifié vous-même et ce que vous avez corrigé. L'important est que vous puissiez expliquer et vérifier chaque décision.